

Table of Contents

Project 1: O-RAN Network Defender	1
Project Summary	1
Learning Objectives	1
Problem Description	2
Game Concept	3
Technical Requirements	4
Controlled Use of AI	6
Code Quality Requirements	6
Security Requirements	7
Scalability Requirements	7
Testing Requirements	8
Functional Acceptance Criteria	8
Stretch Goals	10
Expected Deliverables	11
Final Demonstration	11
Definition of Done	12

Project 1: O-RAN Network Defender

Project Summary

O-RAN Network Defender is a browser-based multiplayer strategy game where teams operate a simplified 5G network management system. Players must monitor simulated network cells, detect faults, respond to incidents, and keep the network healthy while competing or collaborating with other players.

The game is inspired by real mobile network management scenarios, including service degradation, alarm storms, resource contention, misconfiguration, and automated remediation.

The goal is to build a fun but technically serious application that uses the technologies covered in the graduate onboarding programme, with a strong focus on software quality, test automation, security, scalability, and responsible AI-assisted development.

Learning Objectives

By completing this project, graduates should be able to:

1. Build a browser-based multiplayer application using Java, Spring Boot, and modern frontend technologies.
2. Design a clean backend architecture using appropriate design patterns.
3. Implement microservice-style APIs with clear separation of responsibilities.
4. Use Docker and Kubernetes to package, deploy, and scale the application.
5. Apply automated testing across unit, integration, API, and end-to-end levels.
6. Use Python for simulation, data generation, or game event modelling.
7. Use SonarQube to measure and improve code quality, maintainability, security, and test coverage.
8. Use AI tools such as Kiro CLI and IDE in a controlled, reviewable, engineering-led way.
9. Demonstrate secure coding practices, especially around user input, authentication, authorization, and API design.
10. Explain how the solution could scale to support more players, more simulated network elements, and higher event volume.

Problem Description

Mobile network operators need to manage complex 5G networks containing many distributed components. In O-RAN environments, the Service Management and Orchestration layer is responsible for monitoring, configuring, and optimizing network functions.

In this project, your team will build a simplified game version of this problem.

Players are network operations engineers responsible for keeping a virtual 5G network running. The network is made up of multiple cells, each with health indicators such as:

- Signal quality
- User load
- Latency
- Packet loss
- Alarm count
- Energy usage
- Configuration status

During the game, random incidents occur. Examples include:

- A cell becomes overloaded.
- A configuration change causes degraded performance.
- A transport link becomes unstable.
- An alarm storm hides the real root cause.
- A neighbouring cell starts interfering.
- A software upgrade introduces a fault.
- A rogue automation action makes the network worse.

Players must investigate incidents and apply remediation actions. Correct actions improve the network score. Poor actions may increase instability, cost, or customer impact.

The game should be playable from a browser by multiple users at the same time.

Game Concept

Game Mode

The game may be implemented as either:

- **Co-operative mode:** all players work together to keep the network healthy.
- **Competitive mode:** each player or team manages a separate region of the network.
- **Hybrid mode:** players work together on shared incidents but are scored individually.

The team should choose one primary mode for the minimum viable product.

Example Gameplay Loop

1. A player joins a game session from the browser.
2. The backend creates or assigns the player to a simulated network region.
3. Network metrics begin changing over time.
4. Incidents appear as alerts or alarms.
5. Players inspect cell details and choose actions.
6. Actions affect the network state.
7. The game calculates score based on uptime, stability, response time, and action quality.
8. At the end of the game, players receive a performance summary.

Example Incident

Incident: Cell-17 is showing high latency and packet loss.

Available evidence:

- User load is high.
- Neighbouring Cell-18 recently changed configuration.
- Alarm count increased in both cells.
- Energy usage is normal.

Possible player actions:

- Rebalance traffic to neighbouring cells.
- Restart the affected cell.
- Roll back the recent neighbour configuration change.
- Ignore the alarm.
- Increase transmit power.

Expected best action:

Investigate Cell-18 and roll back the configuration change, then rebalance traffic if needed.

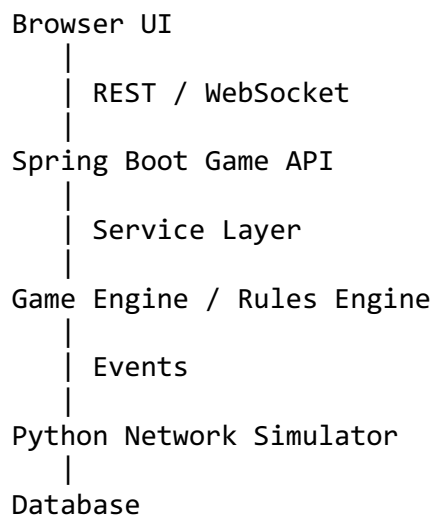
Technical Requirements

Application Architecture

The solution should include:

- A browser-based frontend.
- A Java Spring Boot backend API.
- A real-time or near-real-time game update mechanism.
- A persistence layer for users, game sessions, scores, incidents, and actions.
- A Python-based simulation or incident generation component.
- Docker packaging for all major services.
- Kubernetes deployment manifests.
- SonarQube analysis as part of the development workflow.

A suggested architecture is:



The exact architecture may vary, but the team must be able to explain and justify their design.

Required Technologies

Backend

The backend must use:

- Java
- Spring Boot
- REST APIs

- WebSocket, Server-Sent Events, or polling for live game updates
- JPA, JDBC, or another suitable persistence approach
- Bean validation
- Structured logging
- Automated tests

Frontend

The frontend must provide:

- Browser-based gameplay
- Game session view
- Network map or cell dashboard
- Incident list
- Player action controls
- Scoreboard
- Game summary view

The team may choose the frontend framework, for example React, Angular, Vue, or a server-rendered approach.

Python

Python must be used for at least one meaningful part of the system, such as:

- Network metric simulation
- Incident generation
- Test data generation
- Game balancing scripts
- Score analysis
- Replay analysis

Python should not be included just for the sake of using Python. It should have a clear engineering purpose.

Docker

The solution must include Docker support for:

- Backend service
- Frontend service, if separately hosted
- Python simulator, if separately deployed
- Database, if using a containerized local database
- SonarQube, if running locally as part of the development environment

Kubernetes

The solution must include Kubernetes manifests or Helm-style configuration for:

- Application deployment
- Service exposure
- ConfigMaps or Secrets where appropriate
- Resource requests and limits
- Horizontal scaling for at least one service
- Readiness and liveness probes

Controlled Use of AI

AI tools may be used, but not as a replacement for engineering judgement.

The team must maintain an **AI Usage Log** documenting where AI was used and how the output was reviewed.

Permitted uses include:

- Generating initial design alternatives.
- Explaining unfamiliar APIs or frameworks.
- Creating first drafts of test cases.
- Suggesting refactoring options.
- Reviewing code for readability.
- Generating example data.
- Helping write documentation.

Not permitted:

- Blindly accepting generated code.
- Committing code that nobody on the team understands.
- Using AI-generated security logic without review.
- Using AI-generated tests without checking that they test meaningful behaviour.
- Using AI to bypass design thinking.
- Using AI to create large unreviewed code blocks.

Every AI-assisted code change must be reviewed by at least one human team member.

Code Quality Requirements

The project must use SonarQube or SonarCloud to measure quality.

The team should define and enforce a quality gate that includes:

- No critical or blocker issues.
- Meaningful unit test coverage.
- No obvious code duplication.

- Maintainability rating within an agreed threshold.
- Security hotspots reviewed.
- Clear naming and readable code.
- No hardcoded secrets.
- No unused major code paths.

The team should be ready to explain:

- What issues SonarQube found.
- Which issues were fixed.
- Which issues were accepted and why.
- How test coverage changed during the project.

Security Requirements

The application must include basic security controls appropriate for a multiplayer browser application.

Minimum expectations:

- Input validation on API requests.
- Server-side validation of player actions.
- Users must not be able to modify another player's score directly.
- Game state changes must happen through controlled backend logic.
- No secrets committed to source control.
- Safe handling of configuration values.
- Protection against obvious injection risks.
- Clear authorization rules for joining or managing game sessions.
- Error messages must not leak sensitive implementation details.

Authentication can be lightweight for the project, but the security model must be explicit.

Scalability Requirements

The application should be designed to handle growth beyond a single player and a single game session.

Minimum expectations:

- Multiple simultaneous players.
- Multiple active game sessions.
- Backend designed so game state is not tightly coupled to one browser session.
- Clear explanation of how the system would scale horizontally.
- Kubernetes deployment includes at least one scalable service.
- Avoid unnecessary synchronous blocking where event-based processing would be more appropriate.

- Database access should be designed with reasonable query patterns.

The team should be able to explain what would break first if the game had:

- 100 players.
- 1,000 simulated cells.
- 10 game sessions running at once.
- Frequent incident updates every few seconds.

Testing Requirements

Backend Testing

- Unit tests for game rules.
- Unit tests for scoring logic.
- Unit tests for incident handling.
- Controller or API tests.
- Integration tests involving persistence.
- Negative tests for invalid player actions.

Frontend Testing

- Tests for key UI components.
- Tests for action submission.
- Tests for displaying incidents and scores.
- Basic browser or end-to-end test for the main game flow.

Python Testing

- Tests for the simulator or data generation logic.
- Repeatable simulation output where appropriate.
- Validation that generated incidents are valid.

System Testing

At least one automated test should prove that:

1. A game session can be created.
2. A player can join.
3. An incident can be generated.
4. The player can submit an action.
5. The score or game state changes correctly.

Functional Acceptance Criteria

Game Session

- A user can create a new game session.
- A user can join an existing game session.

- At least two players can participate in the same session.
- The game has a clear start and end condition.

Network Simulation

- The game contains multiple simulated network cells.
- Each cell has visible health metrics.
- Metrics change over time.
- Incidents are generated during gameplay.
- Incidents have enough information for players to make decisions.

Player Actions

- Players can inspect incidents.
- Players can choose from multiple remediation actions.
- Player actions affect the game state.
- Incorrect actions can have negative consequences.
- The same action should not always be the best choice.

Scoring

- The game calculates a score.
- Scoring considers at least three factors, such as:
 - Network health
 - Incident response time
 - Correctness of actions
 - Customer impact
 - Cost of remediation
- A scoreboard is visible in the browser.

Browser Interface

- The game can be played from a browser.
- The UI shows the current network state.
- The UI shows active incidents.
- The UI allows players to submit actions.
- The UI shows scores or team performance.
- The UI provides feedback after actions.

Backend API

- The backend exposes clear APIs for game creation, joining, state retrieval, and action submission.
- Server-side rules determine the outcome of actions.
- Invalid actions are rejected safely.
- API errors are handled clearly.

Deployment

- The application can be run locally using Docker.
- Kubernetes manifests are provided.
- The team can explain how to deploy the system.
- The system has basic health checks.

Quality and Testing

- Automated tests can be run by a single command.
- SonarQube analysis can be run.
- The project meets the agreed quality gate.
- Test results and quality results are documented.

Stretch Goals

Gameplay Stretch Goals

- Add different player roles, such as:
 - Incident Commander
 - Radio Optimization Engineer
 - Automation Engineer
 - Security Analyst
- Add hidden root causes that require investigation.
- Add incident chains where one fault causes another.
- Add a replay mode showing key decisions after the game.
- Add a leaderboard across multiple game sessions.
- Add difficulty levels.

O-RAN / Telecom Stretch Goals

- Model simplified O-RAN components such as:
 - SMO
 - Non-RT RIC
 - Near-RT RIC
 - O-CU
 - O-DU
 - O-RU
- Add simplified policy-based optimization.
- Add xApp or rApp-inspired automation actions.
- Simulate configuration drift.
- Simulate SLA breaches.

Technical Stretch Goals

- Use message queues for incident events.
- Add distributed tracing.

- Add Prometheus metrics.
- Add Grafana dashboards.
- Add load testing.
- Add blue-green or rolling deployment configuration.
- Add feature flags.
- Add a dedicated rules engine.

AI Stretch Goals

- Use AI to generate incident explanations for players, but only after the game engine has determined the actual cause.
- Add an AI assistant that gives hints, but sometimes with limited confidence.
- Require players to judge whether the AI recommendation is safe.
- Log AI suggestions and player decisions for post-game review.
- Add guardrails so the AI cannot directly change game state.

Expected Deliverables

Each team should deliver:

1. Source code repository.
2. Browser-playable application.
3. Docker configuration.
4. Kubernetes deployment files.
5. Automated test suite.
6. SonarQube quality report.
7. Short architecture document.
8. AI Usage Log.
9. Security notes.
10. Final demonstration.

Final Demonstration

The final demo should show:

1. A player creating or joining a game.
2. Multiple players interacting with the same session.
3. Network metrics changing.
4. At least one incident being generated.
5. A player investigating and responding to the incident.
6. The game state and score updating.
7. Tests running successfully.
8. SonarQube results.
9. Docker or Kubernetes deployment.
10. Explanation of where AI was used and how quality was protected.

Definition of Done

The project is considered done when:

- The game is playable from a browser.
- Multiple players can participate.
- Incidents occur and can be resolved.
- Scores update correctly.
- The application runs in Docker.
- Kubernetes deployment files exist.
- Automated tests are present and passing.
- SonarQube quality gate is met or justified.
- Security and scalability considerations are documented.
- AI usage is controlled, logged, and reviewed.